

MB 531: Business Intelligence for Decision Modelling

Assignment 2: Nirvana Homes, Price It Right, Sell It Fast

Google Colab code:

<https://colab.research.google.com/drive/1sS89EeK4W7ynzmbxJF0xfJWzFYPXW9H?usp=sharing>

Setup: 1,400 listings after removing duplicates; missing amenity_score filled with the median (4.8); categories one-hot encoded (base: Bengaluru, tier 1, Fully-Furnished). 80/20 train/test split, random_state = 42 (280 test listings). Only the 17 columns known at listing time were used as features. For the sales-speed models, "slow" (class 0) is the positive class (790 slow, 610 fast).

Question 1: Pricing model

Technique: multiple linear regression. The target (price) is continuous, and Ananya also wants to know how much each feature changes the price, which a regression coefficient tells directly. A regression tree can predict price but gives no "Rs Y lakh per sq ft" number.

Metric	Train	Test
R ²	0.8937	0.9034
RMSE (lakhs)	13.56	12.50
MAE (lakhs)	8.66	8.69

Feature	Coefficient	Feature	Coefficient
locality_tier_3	-43.70	area_sqft	+0.0718
locality_tier_2	-27.57	amenity_score	+1.26
city_Hyderabad	-16.55	parking_spots	+2.56
city_Pune	-11.46	is_corner_unit	+3.80
bhk	-6.08	age_years	-0.70
furnishing_Unfurnished	-4.65	distance_to_metro_km	-1.52
furnishing_Semi-Furnished	-4.16	festival_season_listing_1	-0.53

Intercept 39.73 lakhs. Base: Bengaluru, tier 1, Fully-Furnished. (bkh is negative only because area is already in the model.)

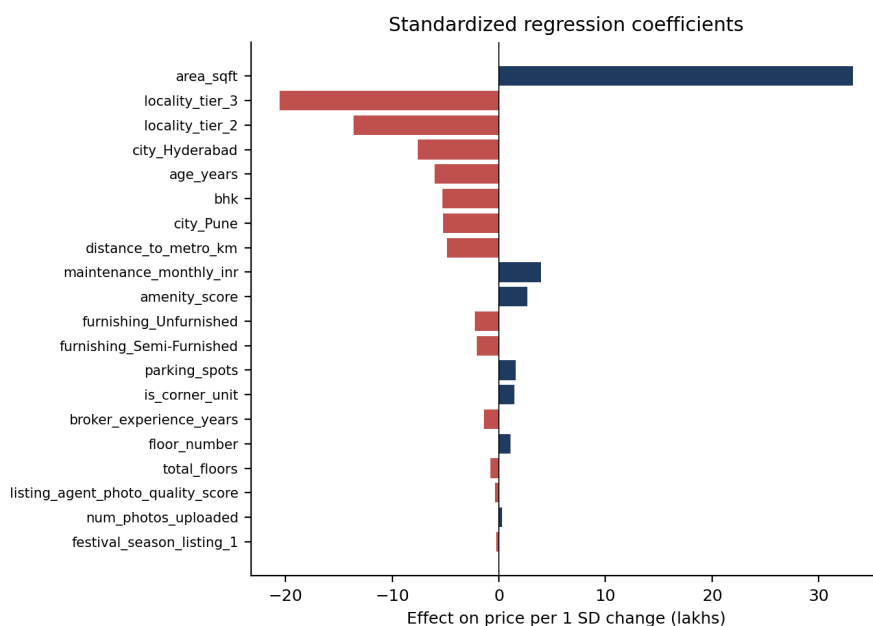


Figure 1. Standardised coefficients (price change per 1 standard deviation).

Dear Ananya, the model explains about 90% of price variation on unseen listings (test R^2 0.903) with an average error of Rs 8.7 lakh (about 11.5% of the mean price), and train and test errors are nearly equal. The two biggest drivers are **area** (about Rs 7,200 more per extra sq ft, or Rs 7.2 lakh per 100 sq ft) and **locality tier** (tier 3 is about Rs 43.7 lakh cheaper than tier 1, tier 2 about Rs 27.6 lakh cheaper). I think your three-comparable method goes wrong because the comparables differ in size, tier, corner, furnishing and metro distance all at once, and a gut-feel tweak cannot fix that (e.g. 200 sq ft is about Rs 14 lakh, a corner unit +3.8 lakh, unfurnished -4.65 lakh, each km from metro -1.5 lakh). Photo score, photo count and festival season barely affect price, so I recommend using the model price as the starting price and not adjusting for them.

Question 2: Where is the model most wrong?

Technique: residual analysis on the 280 test listings (error = actual - predicted, so positive means under-priced). I checked the ten biggest errors and the MAE for each city, tier, BHK, large flats (area of 1,507 sq ft or more, top quartile) and old buildings (20 years or more).

Listing	City	Tier	BHK	Area	Age	Actual	Predicted	Abs. error
NH-00405	Bengaluru	1	4	2191	7	276.11	186.07	90.04
NH-00608	Hyderabad	3	4	2471	12	78.10	129.41	51.31
NH-00704	Bengaluru	2	4	2270	9	212.24	162.87	49.37
NH-00554	Pune	3	3	1686	22	50.08	93.83	43.75
NH-01296	Bengaluru	1	3	1909	11	208.47	166.95	41.52
NH-00511	Bengaluru	1	4	2137	8	225.90	185.20	40.70
NH-01188	Bengaluru	3	4	1941	27	73.51	100.76	27.25
NH-00697	Bengaluru	2	4	2165	7	179.85	154.69	25.16
NH-01333	Pune	1	4	2467	13	209.49	185.40	24.09
NH-00175	Pune	2	4	2434	3	196.43	173.38	23.05

Table: ten largest absolute errors (test set).

Category	N	MAE	RMSE	Mean error
City: Bengaluru	123	8.59	13.63	-0.26
City: Hyderabad	68	8.68	11.80	-1.71
City: Pune	89	8.83	11.33	-0.14
Locality tier 1	67	11.11	16.73	-1.36
Locality tier 2	111	6.46	9.10	+1.58
Locality tier 3	102	9.52	12.45	-2.40
BHK 1	40	9.61	11.35	+3.06
BHK 2	108	6.21	7.77	-0.79
BHK 3	107	8.53	11.01	-3.27
BHK 4	25	18.58	27.64	+6.05
Large flat: Yes	70	14.08	20.16	-1.73
Large flat: No	210	6.89	8.54	-0.19
Old building: Yes	69	9.23	11.66	-1.91
Old building: No	211	8.51	12.76	-0.14
Corner unit: Yes	50	8.12	15.15	+1.53

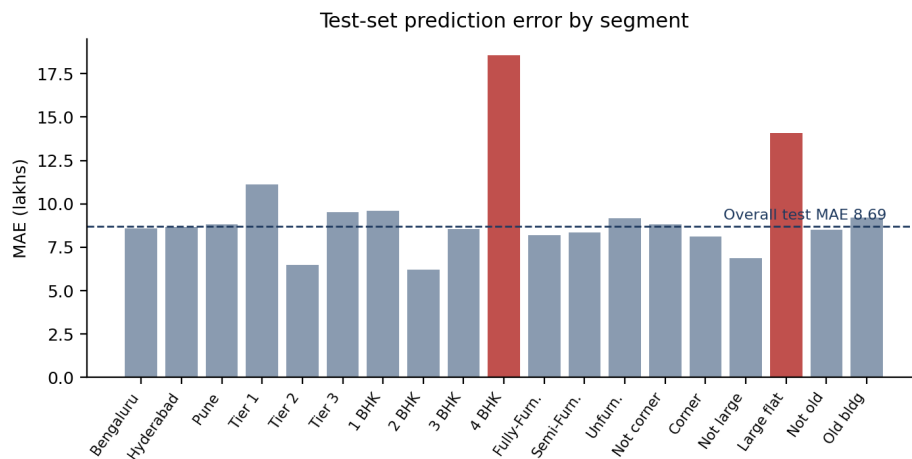


Figure 2. MAE by segment (red = clearly above the overall test MAE of 8.69).

Dear Ananya, the model is not equally reliable: for normal 1 to 3 BHK flats under about 1,500 sq ft the typical error is Rs 6 to 7 lakh, but for 4 BHK (MAE 18.6) and large flats (MAE 14.1) it is two to three times higher, and all ten worst misses are large flats (eight are 4 BHK). The misses go both ways: large tier 1 and 2 flats are under-priced while the three over-priced ones are all tier 3, so one price per sq ft does not fit every tier (errors in lakhs also grow with price, and the 4 BHK group has only 25 listings). I would not use it with no human check; it can give the default price for standard flats, but brokers should review 4 BHK, large (1,500 sq ft or more) and tier 1 listings. City, furnishing, corner unit and age caused no extra error, and adding log(price) or area x tier interactions should help the weak spots.

Question 3: Sales-speed classification

Leakage: these columns are only known after the sale, so they cannot be used.

Column	Problem if left in
sold_within_45_days	It is the target, so accuracy would be 100% and meaningless.
days_on_market	The target is made from it (45 days or fewer). The tree would just split at 45 and score near perfect, but a new listing has no days on market yet, so the model could not be used and the score would be fake.
discount_from_listing_pct	Known only when the seller accepts an offer. Slow listings usually give bigger discounts, so it uses the outcome to predict the outcome.
final_sale_price_lakhs	Known only at sale (asking price minus discount), so it carries the same problem.

Technique: classification tree (max_depth = 5), because the target is yes/no and the rules are easy for brokers to read. It is compared with a majority-class baseline that always predicts "slow".

Metric	Majority baseline	Decision tree
Accuracy	0.568	0.682
Precision (slow)	0.568	0.688
Recall (slow)	1.000	0.805
F1 (slow)	0.724	0.742

Confusion matrix (rows = actual)	Predicted slow	Predicted fast
Tree: actual slow (159)	128	31 (false negatives)
Tree: actual fast (121)	58 (false positives)	63
Baseline: actual slow (159)	159	0
Baseline: actual fast (121)	121	0

Dear Vikram, the tree beats the always-slow baseline by about 11 points of accuracy (68.2% vs 56.8%) and has a recall of 80.5%, so it catches 128 of the 159 genuinely slow listings and is right 68.8% of the time when it flags one. The baseline's 100% recall is meaningless because it flags everything. A false negative is a slow listing the model calls fast (31 of 159, about 1 in 5): it gets no extra attention, sits unsold for months and delays the commission. A false positive (58 fast listings flagged) only wastes some broker time. Since it flags about 2 out of 3 listings, I would use it to order the follow-up list rather than as a short list.

Question 4: Random forest alternative

Technique: random forest (100 trees, default settings). A single tree is unstable, and averaging many trees usually reduces that and improves accuracy. Same split and metrics as the tree.

Metric	Decision tree	Random forest
Accuracy	0.682	0.643
Precision (slow)	0.688	0.661
Recall (slow)	0.805	0.761
F1 (slow)	0.742	0.708

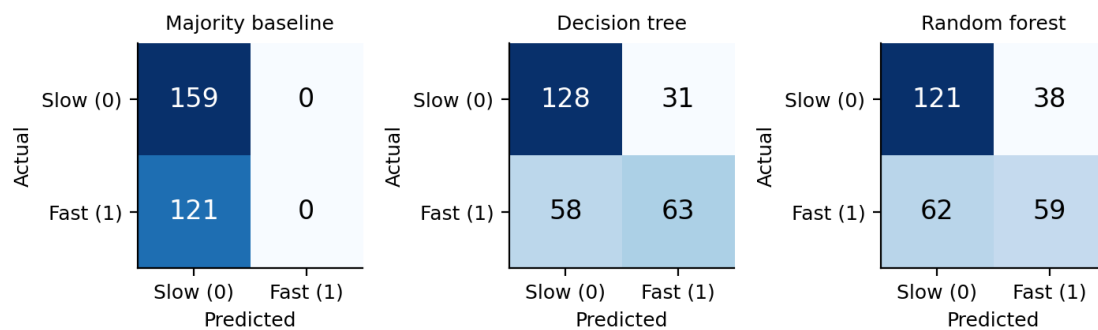


Figure 3. Confusion matrices (baseline, decision tree, random forest).

The forest did not beat the tree. With only 280 test rows, a 4 point gap is within sampling noise (about plus or minus 5.5 points), so the two models perform about the same.

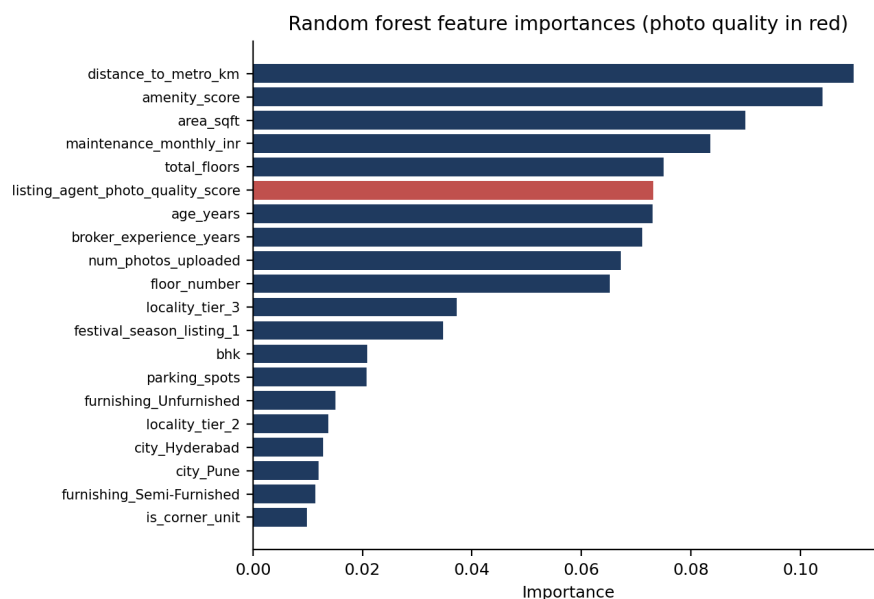


Figure 4. Random forest feature importances (photo quality in red).

Rank	Feature	Importance
1	distance_to_metro_km	0.110
2	amenity_score	0.104
6 (of 20)	listing_agent_photo_quality_score	0.073

Dear Vikram, the data does not show that photo quality is a real driver of fast sales: it ranks 6th of 20 (0.073), but that is close to ordinary features like building age and broker experience, the clear leaders are metro distance, amenity score, area and maintenance, and it had almost no effect on price in Question 1. Importance is not proof of cause, so I would pause more photo spending until a test (better photos on a random half of similar listings) shows it speeds up sales. I would give your team the **decision tree**: it catches slightly more slow listings (80.5% vs 76.1%), its rules are easy to read, and the forest is not better. Use it to decide this week's calls, remembering that about 1 in 5 slow listings will still be missed.

Limitations: one train/test split only (cross-validation would be better); no tuning; and listed_price_lakhs, which is known at listing time, was not used in the sales-speed models and could help.